

Week 2

Data Modeling
Entity Relationship Diagrams

Today's Schedule

- Basics of Data Modeling
- Types of Data Models
- ER Diagram Basics
- Exercise on ER diagrams

What is Data Modeling?

- Data modeling is the process of defining and organizing data in a structured way so it can be stored and used efficiently in a database.
- Ensures the database supports the needs of the system.

Purpose of Data Modeling

- To understand data requirements of a system.
- To define **entities (things)**, their **attributes (properties)**, and **relationships (connections)**.

Types of Data Models

1. Conceptual Data Model

- High-level view: ER Diagram(entities, attributes & relationships)
- Audience: business stakeholders, analysts

2. Logical/Representational Data Model

- Detailed view: tables, columns, keys, relationships
- Focus on the design part of database
- Technology-neutral

3. Physical Data Model

- Implementation view: datatypes & constraints
- Ready to run in DBMS

- **Data modeling is like designing a house.**
- Conceptual → rough sketch of rooms.
- Logical → blueprint with dimensions and layout.
- Physical → actual construction with bricks and wiring.

Conceptual Data Model Example

- **Entity Relationship (ER) Diagrams**
- **Entities:** Student, Course, Instructor
- **Relationships:**
 - - Student enrolls in Course
 - - Instructor teaches Course
- **Focus:** What are the main concepts?

Logical Data Model Example

- **Tables:**
- Student (StudentID, Name, Email)
- Course (CourseID, Title, Credits)
- Instructor (InstructorID, Name, Department)
- Enrollment (StudentID, CourseID, Grade)
- Focus: What tables and fields do we need?

Physical Data Model Example

- **SQL Implementation:**

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY AUTO_INCREMENT,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE Enrollment (  
    StudentID INT,  
    CourseID INT,  
    Grade CHAR(2),  
    PRIMARY KEY (StudentID, CourseID)  
);
```

Focus: How is it implemented in DBMS?

Summary of Data Models

- Conceptual → High-level concepts (ERD)
- Logical → Tables & relationships (Student, Course, Enrollment)
- Physical → DBMS implementation (SQL
CREATE TABLE)

ER Diagrams

What is an ER Diagram?

- Visual tool used in data modeling.
- Shows entities (things), attributes (properties), and relationships (connections).
- Introduced by Peter Chen (1976).
- Used to design databases before implementation.
- Think of it as the blueprint of a database.

Entities

- Represent real-world objects (nouns).
- **Entity Type**: is a collection of entities that share common properties or characteristics e.g EMPLOYEE.
- **Entity instance**: is a single occurrence of an entity type.
- **Example**: there is one EMPLOYEE entity type in most organizations, but there may be hundreds of instances of this entity stored in the database.

Types of Entities

- **Strong Entity** → exists independently (e.g., Student, Course).
- **Weak Entity** → depends on another strong entity (e.g., OrderItem depends on Order. Order is the owner of OrderItem).
- Weak entities don't have a full primary key on their own.
- Owner entity provides a part of primary key
- **Symbols:**
- Strong entity → Rectangle
- Weak entity → Double rectangle

- **Example: Order and OrderItem**
- **Order** = strong entity (has OrderID as PK).
- **OrderItem** = weak entity (depends on Order).
- **Identifying Relationship** = “Contains” (Order **contains** OrderItems).
- OrderItem’s primary key is usually a **combination**: (OrderID, ItemNo)

Examples of entities:

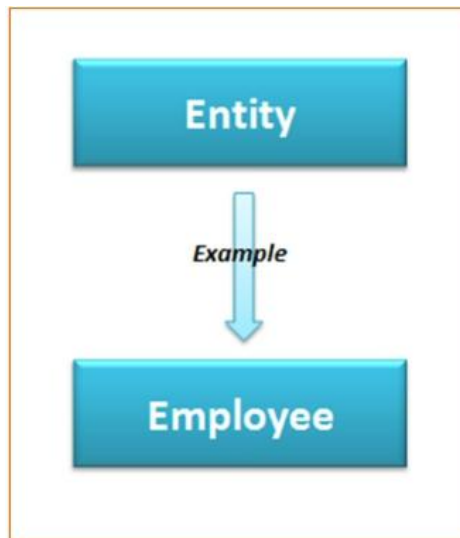
Person: EMPLOYEE, STUDENT, PATIENT

Place: STORE, WAREHOUSE

Object: MACHINE, PRODUCT, CAR

Event: SALE, REGISTRATION, RENEWAL

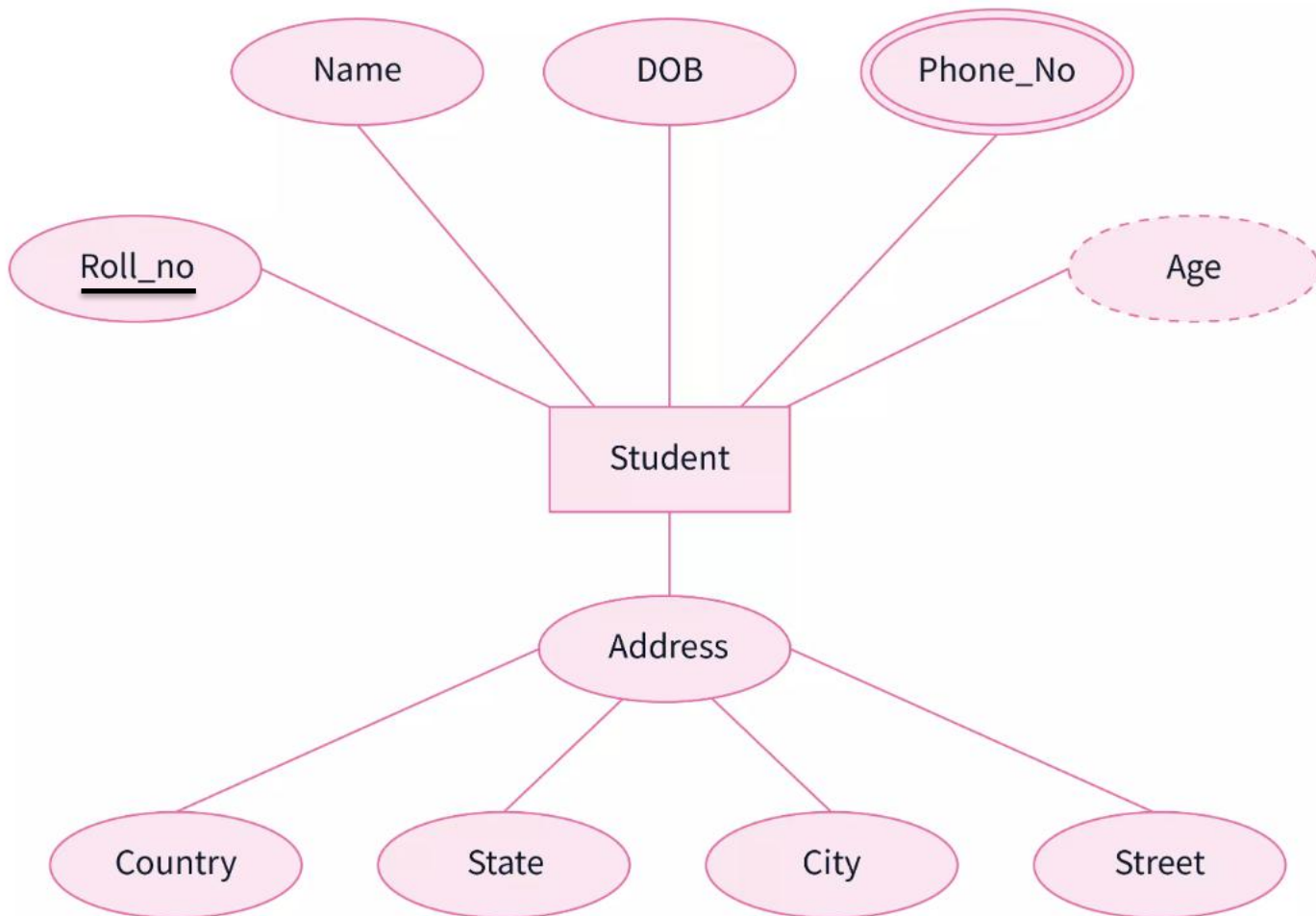
Concept: ACCOUNT, COURSE



Attributes

- Describe properties of an entity.
- Types:
 - **Simple**: cannot be divided (e.g., Age).
 - **Composite**: can be broken down (e.g., Name → FirstName, LastName).
 - **Derived**: can be calculated (e.g., Age from DOB).
 - **Multivalued**: multiple values (e.g., PhoneNumbers).
- Symbols:
- Oval → Attribute
- Double Oval → Multivalued Attribute
- Dashed Oval → Derived Attribute
- Underline → Primary Key (uniquely identifies each entity)

Different types of Attributes



Candidate Key vs Identifier

- **Candidate Key**

Any attribute (or combination) that can uniquely identify a record.

A table can have multiple candidate keys.

Example (Student table): StudentID, Email.

- **Identifier (Primary Key)**

The chosen candidate key used as the main identifier.

Usually called the Primary Key.

Example: StudentID.

Candidate Key = possible options.

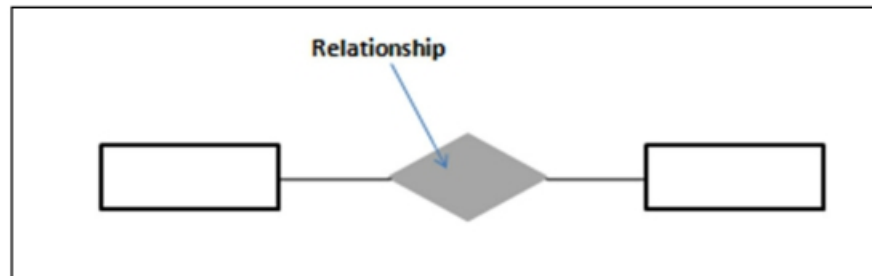
Identifier = the chosen option (Primary Key).

A student can be identified by StudentID, Email, or Passport (candidate keys).

University chooses StudentID as the official identifier.

Relationships

- Show how entities are connected.
- Symbol: Diamond
- Connected to the participating entity types via straight lines



Degree of a Relationship

Definition: Number of entity types that participate in a relationship.

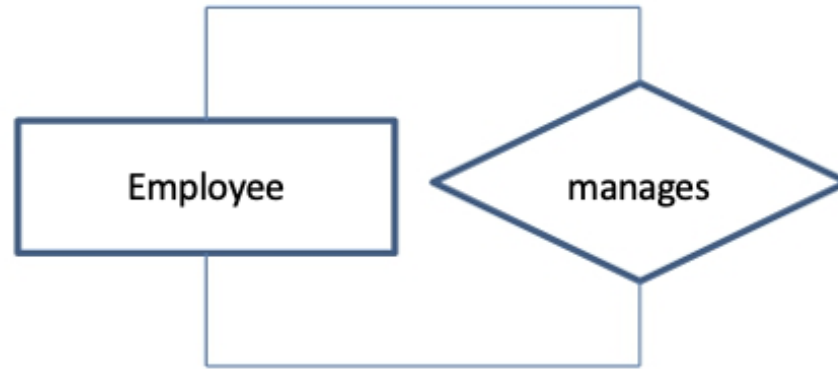
Types:

Unary (degree 1): Relationship within same entity (e.g., Employee manages Employee).

Binary (degree 2): Between two entities (e.g., Student enrolls in Course).

Ternary (degree 3): Among three entities (e.g., Doctor prescribes Medicine to Patient).

Unary



Unary Relationship

between two instances of one entity type

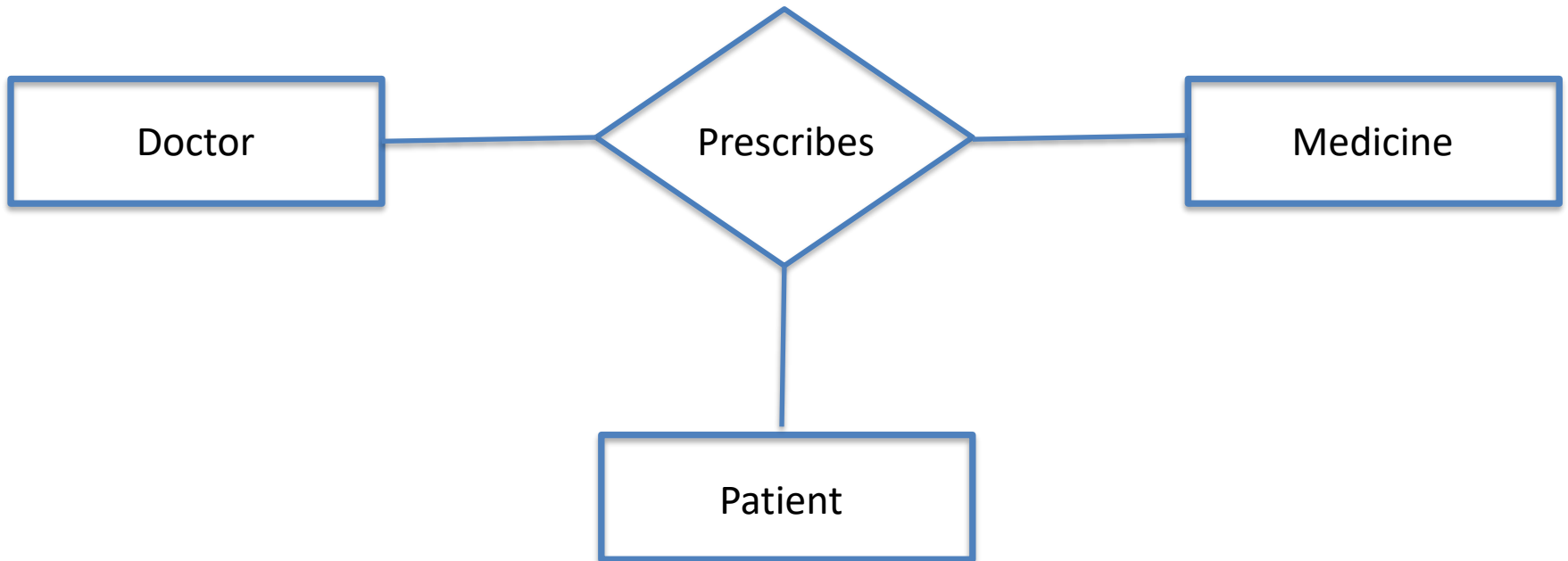
Binary



Binary Relationship

between the instances of two entity types

Ternary

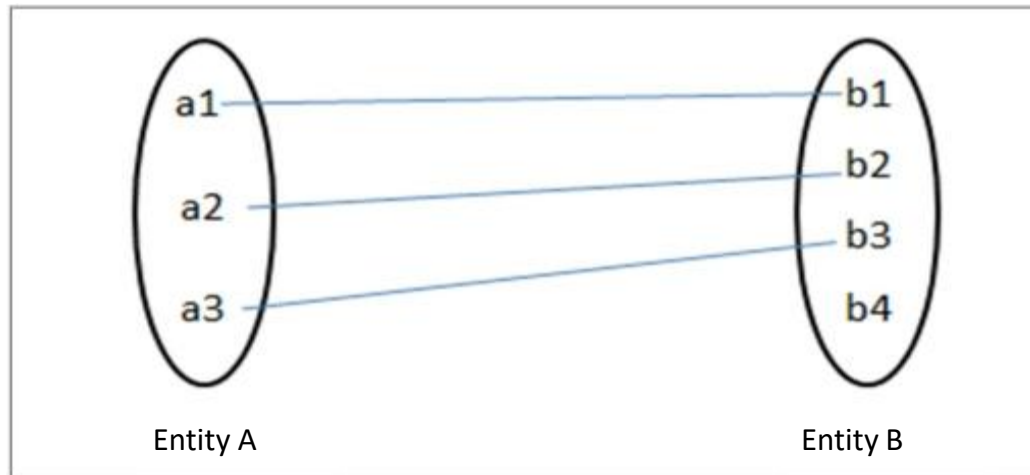


among the instances of three entity types

Cardinality in Relationships

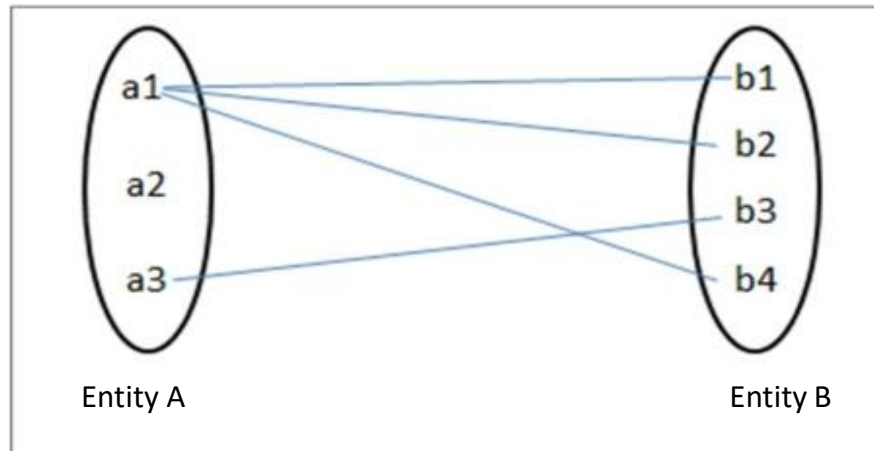
- Definition: Specifies how many instances of one entity can/must be associated with instances of another entity.
- **Types:**
- One-to-One (1:1): Each Student has one Locker.
- One-to-Many (1:N): One Instructor teaches many Courses.
- Many-to-Many (M:N): Students enroll in many Courses, and each Course has many Students.

✎ **1:1 (*one-to-one*)**: one entity instance in A is related to at most one entity instance in B, and one entity instance in B is related to at most one entity instance in A.



Example: Each Student has one Locker.

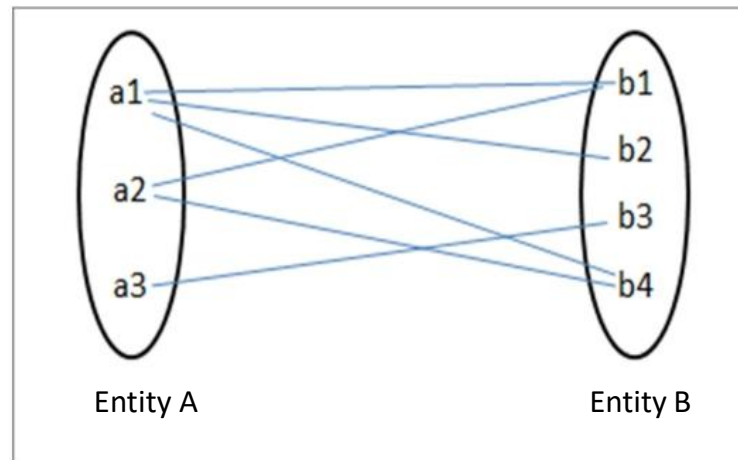
✎ **1:N (*one-to-many*)**: one entity instance in A is related to any number of entity instances in B, but an entity instance in B is related to at most one entity instance in A.



One Instructor teaches many Courses.

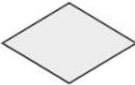
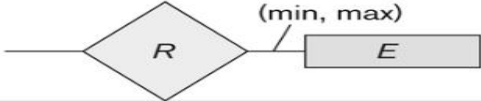


M:N (*many-to-many*): one entity instance in A is related to any number of entity instances in B, and one entity instance in B is related to any number of entity instances in A.

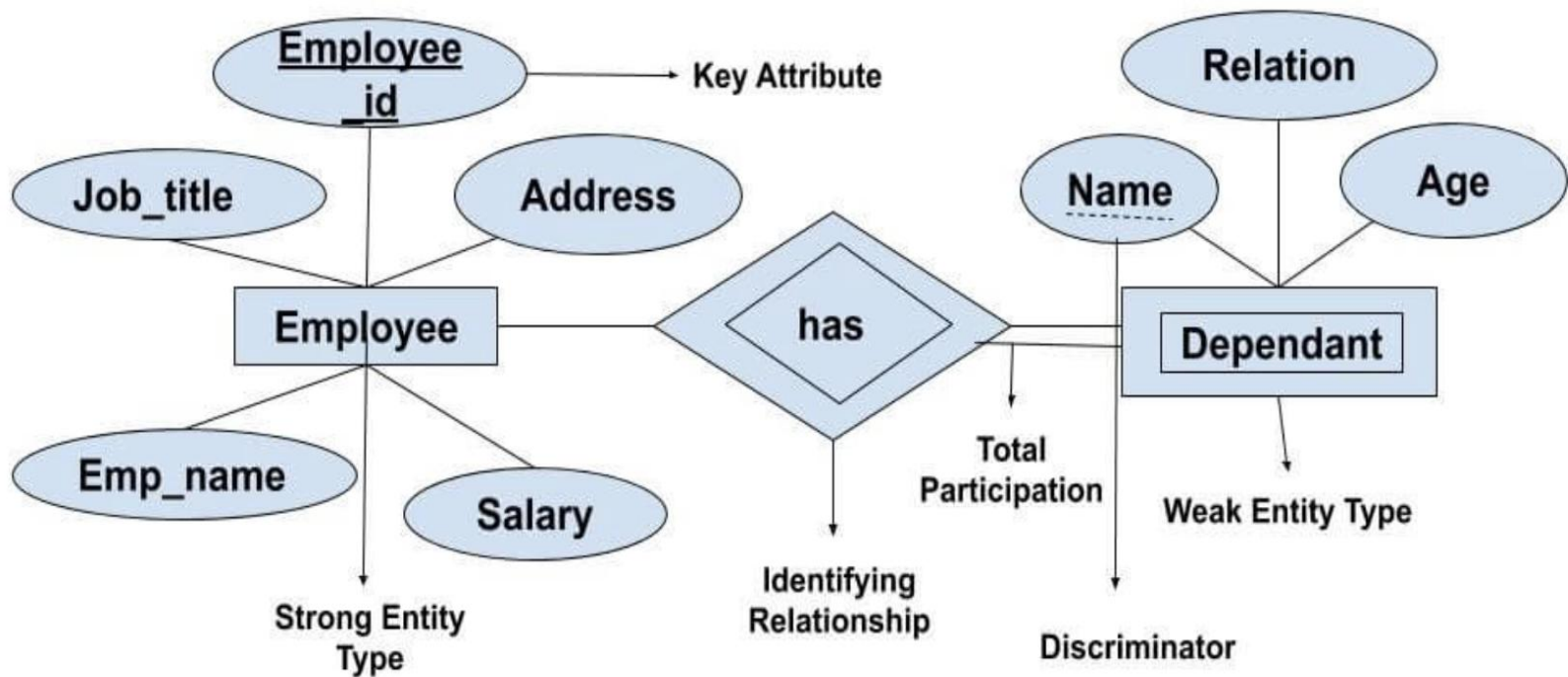


Students enroll in many Courses, and each Course has many Students.

ER Diagram Symbols Summary

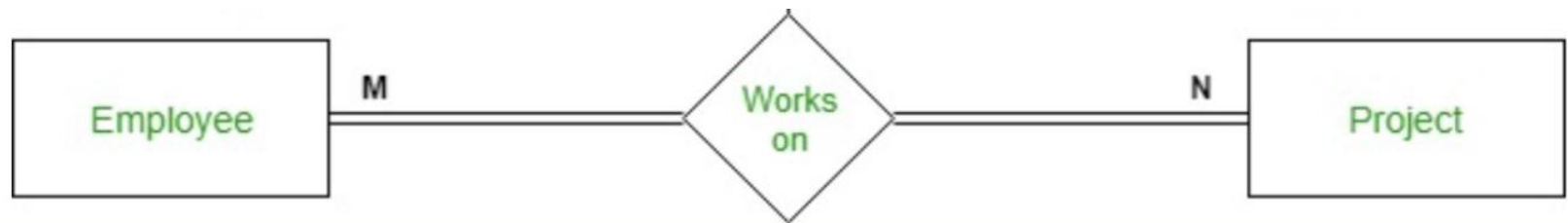
Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1 : N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Weak Entity, Strong Entity, Identifying relationship

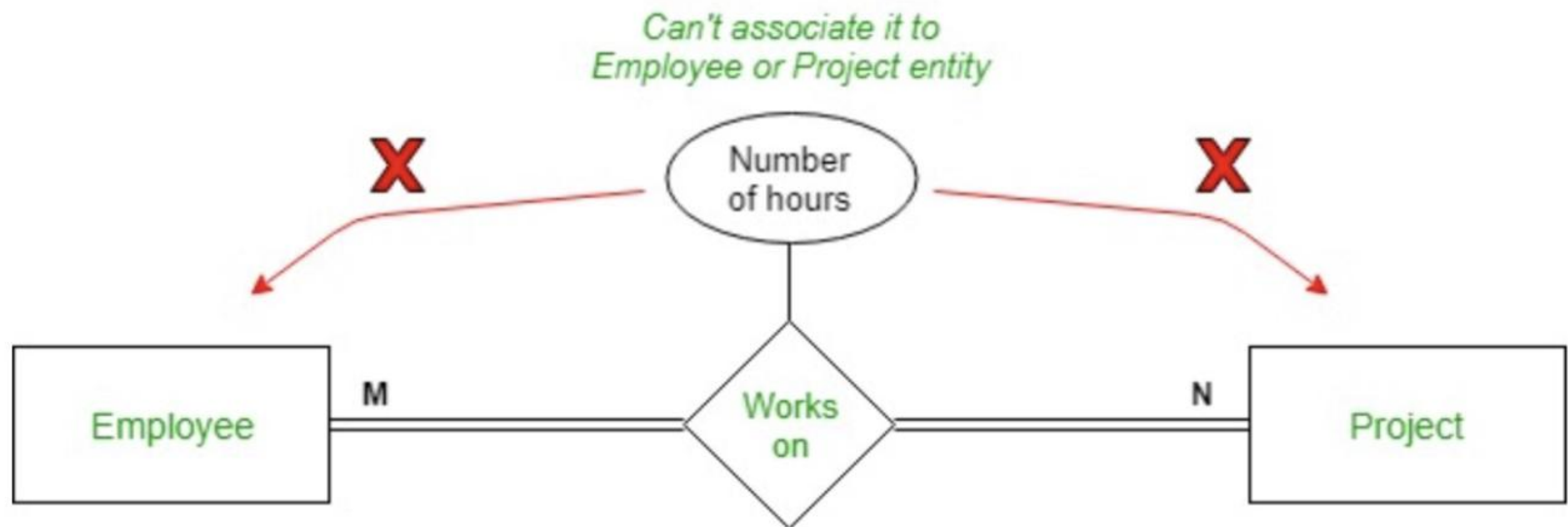


Attributes of Relationship types

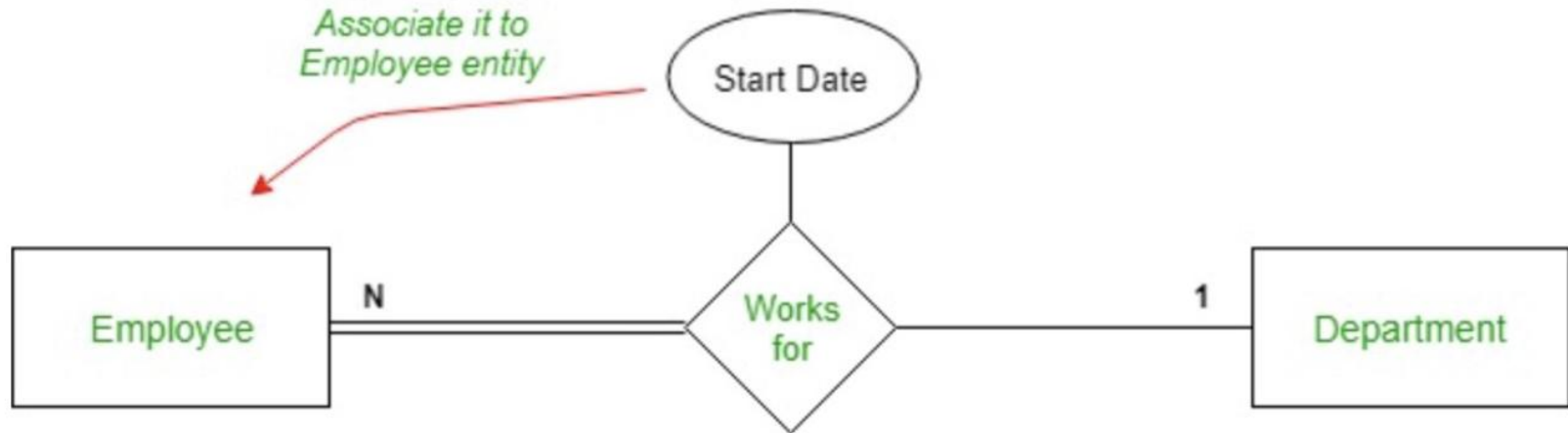
- If an attribute depends on the **combination of two entities**, it belongs to the **relationship**, not the entities individually.
- Most relationship attributes are used with M:N relationships.
- If we want to keep track of **Hours** worked by each employee in each project



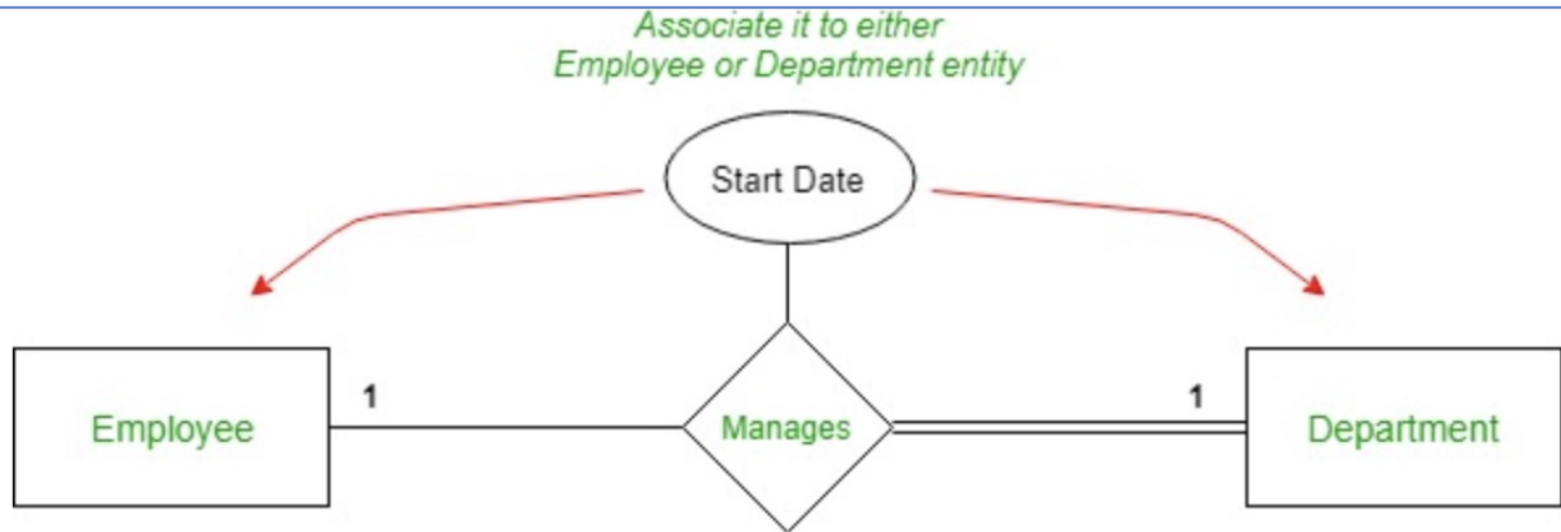
- A value of **Hours** depends on a particular (employee, project) combination.
- It is not just about the employee, and not just about the project → it's about the *employee working on that project*.
- Therefore, it becomes an attribute of relationship



- In 1:N relationships, relationship attributes can be transferred to the entity type on the N-side of the relationship



- In 1:1 relationships, relationship attributes can be transferred to either of the entity or the relationship



Exercise: Company Database

- Please read the description of the COMPANY Database given in the example.
- Identify all the entities (weak & strong) mentioned.
- Identify all the relationships between entities.
- Identify all the attributes mentioned.

Example of Company database Requirements

- The COMPANY database keeps track of a company's employees, departments and projects. The company is organized into departments.
- Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.

Example of Company database Requirements

- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.

Example of Company database Requirements

- We store each employee's name (First name, Middle name, Last name), social security number, address, salary, gender, and birth date.
- An employee works for one department but may work on several projects
- We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's name, gender, birth date, and relationship to the employee.